FUNDAMENTOS DE PROGRAMAÇÃO

SUMÁRIO

VCS 2
GIT 2
INSTALANDO O GIT 4
FLUXO DE TRABALHO 5
COMO USAR 5
PUSH 6
BRANCHES 6
Atualizando e dando merge 7
Conclusão 7
RESUMÃO DOS COMANDOS 8
DICA: FERRAMENTA VISUAL 8

GIT

Em 2005, Linus Torvalds (o homem conhecido por criar o núcleo, ou kernel, do SO Linux) desenvolveu o GIT, que desde então tem sido ativamente mantido por Junio Hamano, um engenheiro de software japonês. Atualmente, o GIT é um dos mais famosos sistemas de controle de versão de código aberto e milhões de projetos no mundo inteiro o utilizam para seu controle de versão (incluindo projetos comerciais e de código aberto). O Git é totalmente grátis e pode ser instalado em Mac, Linux, Windows e Solaris diretamente do site oficial.

https://git-scm.com/downloads

Algumas das características essenciais do GIT são:

- Um sistema de controle de versão distribuído, o GIT segue uma abordagem *peer to peer*, contrário de outros como o Subversion (SVN) que segue um modelo baseado em cliente-servidor.
- GIT permite aos desenvolvedores ter uma infinidade de ramos de código completamente independente. Criação, exclusão e fusão desses ramos é simples e não leva tempo.

VCS

Os sistemas de controle de versão são uma ótima forma de prevenir que você arranque seus cabelos sempre que tentar analisar as alterações feitas em um código de projeto compartilhado.

Simplificando, um VCS (sigla para *version control system*) é um componente fundamental do sistema de gerenciamento de configuração de software que cuida das mudanças que precisam ser feitas em um projeto.

As alterações/revisões/atualizações feitas são identificáveis através de códigos de letra ou números. Informações como data e a identidade do autor da alteração também são mantidas.

- No GIT, todas as operações são atômicas. Isso significa que uma ação pode ter sucesso ou falhar (sem fazer nenhuma alteração). Isso é importante porque em alguns sistemas de controle de versão (como o CVS) onde as operações não são atômicas, se uma operação de repositório é suspensa, ela pode deixar o repositório em um estado instável.
- No GIT, tudo é armazenado dentro da pasta .git. Isso não é o mesmo em outros VCS como SVN e CVS onde os metadasdados de arquivos são armazenados em pastas ocultas (por exemplo, .cvs, .svn, etc.).
- GIT usa um modelo de dados que ajuda a garantir a integridade criptográfica de qualquer coisa presente dentro de um repositório. Cada vez que um arquivo é adicionado ou um commit é feito, suas somas de verificação são geradas. Da mesma forma, eles são recuperados através de suas somas de verificação também.

Outra característica presente no GIT é sua área de teste ou índice. Na área de preparação, os desenvolvedores podem formatar commits e receber feedback antes de aplicá-los.

INSTALANDO O GIT

O GIT é consideravelmente simples de usar. Para começar, você pode criar um repositório/pasta ou utilizar um já existente. Após a instalação, um simples `git init` irá deixar tudo pronto. Da mesma maneira, o comando `git clone` pode criar uma cópia de um repositório local para um usuário.

Instalar o GIT no Windows é tão simples como baixar um instalador e executá-lo. Execute os passos para instalar o GIT no Windows:

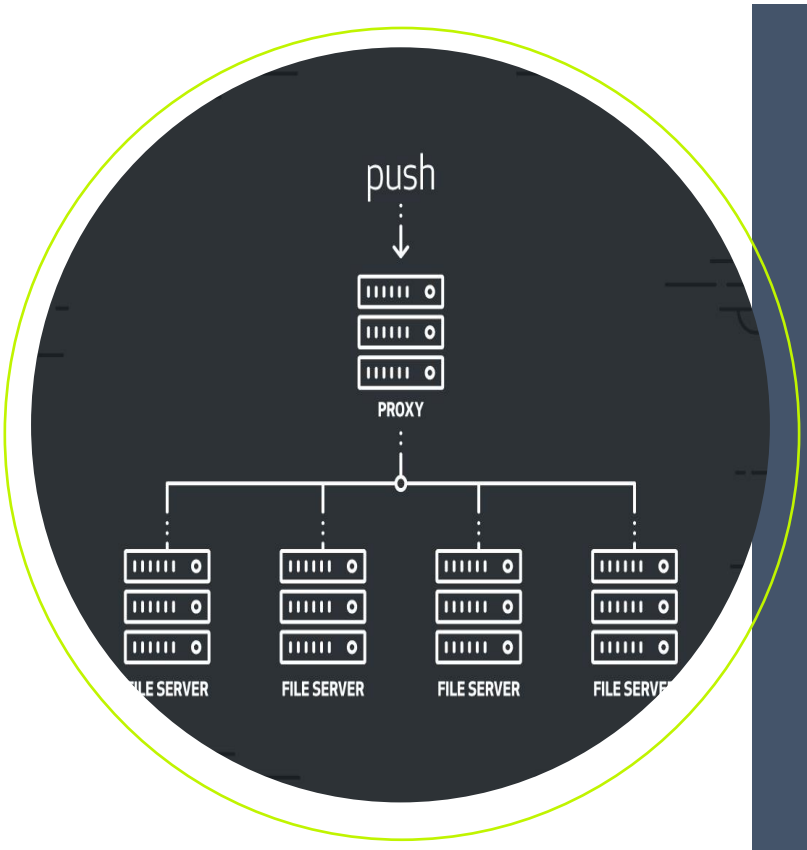
- Acesse o site oficial e faça o download do instalador do GIT para Windows.

- Depois de baixado, clique duas vezes no arquivo para iniciar o assistente de instalação. Basta seguir as instruções na tela, clicando em Next. Ao término, clique em Finish para concluir com êxito a instalação.

Abra o prompt de comando e digite os seguintes comandos no terminal:

```
git config --global user.name "Marcos Santos"
git config --global user.email marcos.santos@imed.edu.br
```

GIT instalado no Windows com sucesso!



FLUXO DE TRABALHO

Agora que o repositório está pronto, vamos falar sobre a estrutura que é mantida pelo GIT. Cada repositório local consiste em três árvores: o diretório de trabalho que contém os arquivos reais; O índice que desempenha o papel de uma área de teste e o HEAD que é um ponteiro para o último commit feito pelo usuário.

Então, é assim que o fluxo de trabalho pode ser explicado: o usuário adiciona um arquivo ou alterações do diretório de trabalho para o índice (a área de teste) e uma vez revistos, o arquivo ou as alterações são finalmente registrados com o HEAD. Alterações ou adições de arquivos propostas são adicionadas ao índice usando o comando add. Para adicionar qualquer arquivo, o comando é:

```
git add <nome_do_arquivo>
```

Se você está realmente confiante o suficiente para fazer essas mudanças no HEAD, então você pode usar o comando commit:

```
git commit -m "Adicionar qualquer mensagem sobre o commit aqui"
```

COMO USAR

Agora que o GIT está instalado e configurado no seu dispositivo, vamos explorar os conceitos básicos do GIT e como começar a usar o GIT a nível de repositório.

Um repositório é o maior bem de qualquer projeto controlado por versão. Para transformar qualquer diretório em um repositório GIT, o simples comando `git init` pode ser utilizado. Uma pasta chamada .git deve começar a existir no diretório em que o comando foi executado.

Por outro lado, se você já tem um diretório e deseja utilizá-lo (clone-o). Você pode usar o comando `git clone`. Se você estiver tentando verificar um repositório local, use o seguinte comando:

```
git clone /path/to/local/repositor
```

Se você pretende verificar um repositório armazenado remotamente, use:

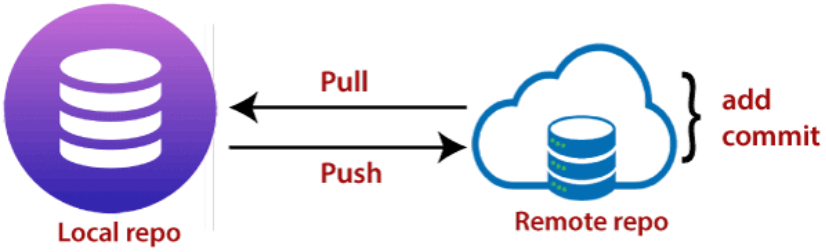
```
git clone user.name@host:/path/to/remote/repository
```

Neste momento que você deve buscar a URL do Clone de seu repositório online, como o github https://github.com/ ou bitbucket https://bitbucket.org/.

Se você não tem uma conta nestes repositórios, crie-a imediatamente.

PUSH

Depois de confirmar as alterações (e acreditar que elas estão prontas para serem enviadas para o repositório original), você pode usar o comando push.



Se, no entanto, um repositório existente ainda não tiver sido clonado e você pretende estabelecer uma ligação entre o seu repositório e um servidor remoto, execute o seguinte comando:

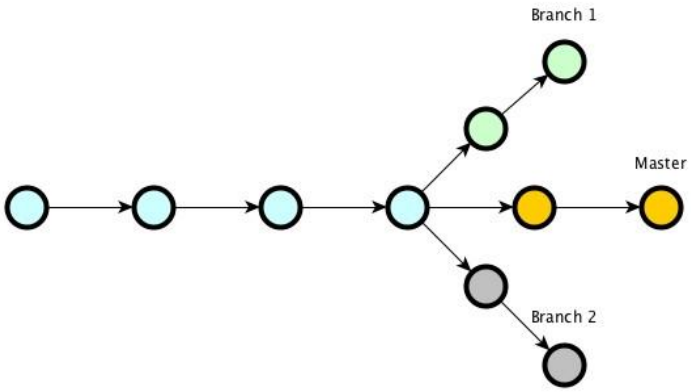
```
git remote add origin <servidor>
```

Uma vez clonado, quaisquer alterações feitas serão aplicadas para o servidor pertinente.

Uma vez que o `git push origin master` é executado de dentro do diretório de trabalho, as mudanças presentes no HEAD são enviadas para o repositório remoto. No comando acima mencionado, o master pode ser alterado para o nome do branch ao qual você deseja que as alterações sejam registradas.

BRANCHES

Outra característica brilhante (mas avançada) do GIT é sua capacidade de permitir que desenvolvedores e gerentes de projeto criem vários ramos (branches) independentes dentro de um único projeto. O objetivo principal de um branch é desenvolver novas funcionalidades, mantendo-os isolados uns dos outros. O branch padrão em qualquer projeto é sempre o **master branch**. Tantos branches quanto necessários podem ser criados e eventualmente mesclados ao master branch, como se fosse colocar os códigos em produção.



Um novo branch pode ser criado usando o seguinte comando:

```
git checkout -b feature_n *
```

feature_n é o nome do branch.

Se você deseja retornar ao master branch, o seguinte comando pode ser usado:

```
git checkout master
```

Qualquer branch pode ser excluído usando o seguinte comando:

```
git checkout -b feature_n
```

Para tornar o branch disponível para outros usuários, você terá que movê-lo para o repositório remoto. Para fazer isso, use o seguinte comando:

```
git push origin feature_n
```

ATUALIZANDO E DANDO MERGE

Caso você queira atualizar seu diretório de trabalho local para o mais recente do repositório remoto, o simples comando `git pull origin master` pode ser usado. Para mesclar outro branch (dar um merge) no atualmente ativo, use:

```
git merge feature_n.
```

Se você der um merge ou pull, o GIT sempre tenta lidar com os conflitos por conta própria, mas as vezes não consegue. Em caso de falha devido a conflitos, o usuário tem que resolver os conflitos manualmente. Depois de editar os arquivos (para erradicar conflitos), marque-os como merged usando:

```
git add <nome.arquivo>
```

Se antes do merge você desejar visualizar as alterações, o seguinte comando pode ser executado:

```
git diff <nome_branch_origem> <nome_branch_alvo>
```

CONCLUSÃO

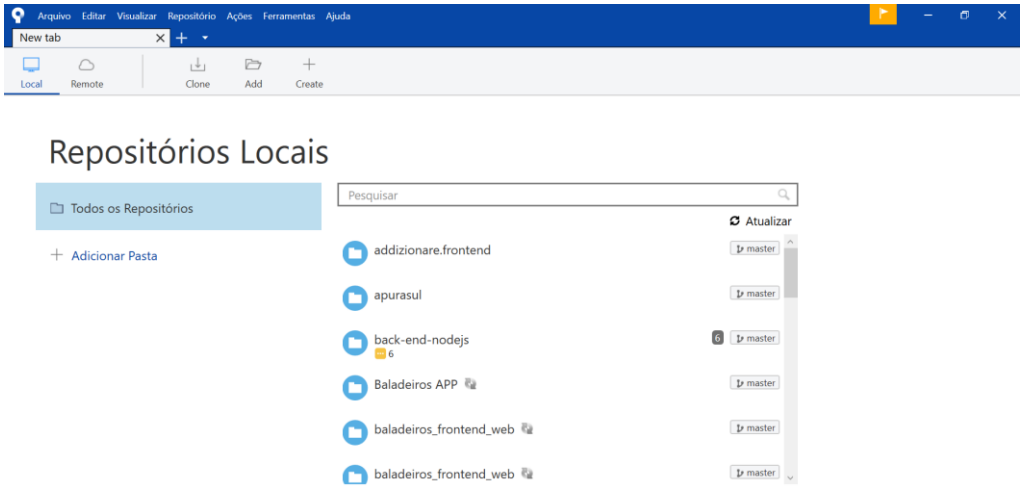
No mundo de projetos de software, é sempre reconfortante saber que alguém está cuidando de toda a gestão de código para você.

Este tutorial básico GIT irá ajudar qualquer desenvolvedor a começar a utilizar o GIT, que é um rigoroso (e muito útil) sistema de controle de versão com uma infinidade de recursos.

RESUMÃO DOS COMANDOS

Verificando a versão
```
git --version
```
Inicializando um Repositório
```
git init
```
Configurando o usuário local
```
git config --global user.name "Marcos Santos"
git config --global user.email marcos.santos@imed.edu.br
```
Adicionando um arquivo para controle
```
git add <nome_do_arquivo>
```
Colocar todos os arquivos
```
git add --all
```
Criando o primeiro Commit
```
git commit -m "V.0"
```
Adicionando um Remote Server
```
git remote add origin https://github.com/profmarcossantos/TesteAula.git
```
Enviando para o servidor os dados
```
git push origin master ou main
```
baixando do servidor e perguntando se aceita as modificações
```
git pull origin master ou main
```
verificando o status do git
```
git status
```
Clonando um Repositório
```
https://github.com/profmarcossantos/TesteAula.git
```
Verificando o histórico de Commits
```
git log
```
#Voltando ao um ponto específico do código (commit)
```
git checkout 53efa8a983f62cb7b42050ec278935e6690e93f2
```
#Retrocendo ao ponto final do Projeto
```
git checkout master ou main
```

DICA: FERRAMENTA VISUAL



Sourcetree